第二章 计算机系统结构基础

1. 性能与执行时间有关，与MIPS与MFLOPS无直接关系
2. 浮点协处理器运行时间 = 定点运行时间（MIPS） + 浮点运行时间（MFLOPS）
3. 计算处理器静态功率时，电阻不变，根据电压求静态电流（为总电流的一部分），再求静态功率。动态功率与翻转频率成正比。总功率 = 静态功率 + 动态功率

第三章 二进制与逻辑电路

1.
   1. **原码**：原码是最直观的表示方法，它直接用二进制数表示一个数，包括正负号。在原码中，最高位（最左边的位）是符号位，0 表示正数，1 表示负数。其余位表示数值本身。例如，十进制数 +5 的原码表示为0000 0101，而 -5 的原码表示为 1000 0101。
   2. **反码**：反码主要用于表示负数。对于正数，其反码与其原码相同。对于负数，其反码是将原码除符号位外的所有位取反（0 变 1，1 变 0）。例如，十进制数-5的反码表示为1111 1010。
   3. **补码**：补码是计算机中最常用的表示方法，用于进行二进制加法运算。对于正数，其补码与其原码相同。对于负数，其补码是其反码加 1。补码的一个重要特性是，任何数的补码加上该数本身，结果总是 0。例如，十进制数 -5 的补码表示为 1111 1011。

   8位补码表示 -2^7 为 `1000 0000`
2. $(-1)^S \times (1.F) \times 2^{E-\text{Bias}}$

情况	阶码 E	尾数 F
±0	全 0	全 0
非规格化数	全 0	非 0
±∞	全 1	全 0
NaN	全 1	非 0

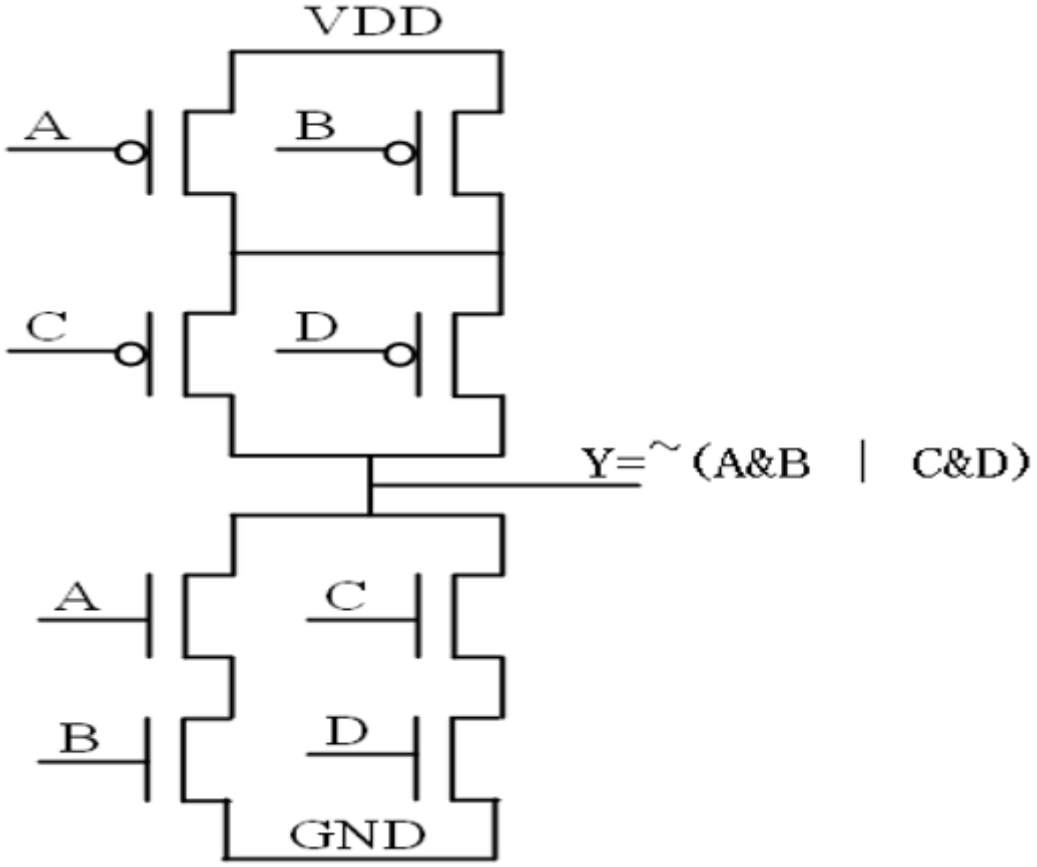
项目	单精度 float	双精度 double
总位数	32	64
尾数位	23	52
阶码位	8	11
Bias	127	1023

3. 用NMOS组成正逻辑：**串联表示与，并联表示并；**

   用PMOS组成反逻辑：~(A*B)=~A+~B，~(A+B)=~A*~B；

   正反逻辑串联： ~(A&B | C&D)；

   最后加一个反相器。



4. 反相器延迟 = 本征延迟（翻转延迟） + 负载延迟（n * （输入电容 + 连线电容） * 延迟系数）
5. 线分析 `C` 与 `CN`，`建立时间` 是 $T_{D-N_2}-(min(C,CN)+T_{trans})$，`保持时间` 是 $(max(C,CN)+T_{trans})-T_{D-N_1}$，`CLK - > Q延迟` 是 $(max(C,CN)+T_{trans})+T_{N_2-Q}$
6. 触发器分析边缘变化瞬间，比如 `0 -> 1`，此时先分析 `0时刻`，各个触发器的输出作为下一时刻 `1时刻` 的输入，共同分析。有时间前后性。

第四章 指令系统结构

1. （1）堆栈型指令系统：堆栈型指令系统又称为**零地址指令系统**。它的操作数都隐含在**栈顶**，在运算指令中不需要显式指定操作数。例如执行加法指令时，系统默认将栈顶的两个操作数相加，并把运算结果重新压回栈顶。该指令系统的特点是指令格式简单，但对栈的访问频繁，数据通路受限。

   （2）累加器型指令系统：累加器型指令系统又称为**单地址指令系统**。它的一个操作数是**隐含的累加器**，另一个操作数由指令显式给出，运算结果通常也写回到累加器中。由于累加器是固定的隐含操作数，指令长度较短，但运算过程中对累加器的依赖较强，灵活性有限。

   （3）寄存器—存储器型指令系统：寄存器—存储器型指令系统中，**每个操作数都由指令显式指定**，操作数可以是寄存器，也可以是存储器单元地址。运算时，一个操作数来自寄存器，另一个操作数来自存储器，运算结果通常存入寄存器。该指令系统减少了部分访存次数，但指令格式相对复杂。

   （4）寄存器—寄存器型指令系统：寄存器—寄存器型指令系统中，**所有操作数都来自寄存器**，每个操作数均由指令显式指定，运算结果也存放在寄存器中（访存仅通过专门的访存指令完成）。该系统的指令执行速度快、数据通路规整，是现代 RISC 处理器常用的指令系统结构，但对寄存器数量要求较高。

	堆栈型	累加器型	寄存器-存储器型	寄存器-寄存器型

汇编代码	Push B
	Push C
	Sub
	Pop A
	Push A
	Push C
	Sub
	Pop D
	Push D
	Push A
	Add
	Pop B
		Load C
		Neg
		Add B
		Store A
		Load C
		Neg
		Add A
		Store D
		Add A
		Store B
			Load R1, B
			Sub R1, C
			Store R1, A
			Sub R1, C
			Store R1, D
			Add R1, A
			Store R1, B
				Load R1, B
				Load R2, C
				Sub R3, R1, R2
				Store R3, A
				Sub R4, R3, R2
				Store R4, D
				Add R5, R4, R3
				Store R5, B

指令字节	30	26	28	29
内存交换字节	48	42	42	39
代码量衡量		√		
交换数据量衡量				√

2. 小端：低地址放低位（右侧，`0x12345678`中的`78`），大端：低地址放高位（左侧，`0x12345678`中的`12`）

3. `MIPS`是以`4字节`对齐的，条件转移的地址范围是 2^{16}，直接跳转的地址范围是 2^{18}，均需要左移`2位`，即`128KB`和`256MB`。

4.

OpCode	Description
LB	Load Byte
LBU	Load Byte Unsigned
LH	Load Halfword
LHU	Load Halfword Unsigned
LW	Load Word
LWL	Load Word Left
LWR	Load Word Right
SB	Store Byte
SH	Store Halfword
SW	Store Word
SWL	Store Word Left
SWR	Store Word Right

边界不对齐的数据传送（小尾端）

初始值

M[100]	A	V	E	
M[104]				D
R2	J	O	H	N

执行“LWR R2, 0x101”后

R2	J	A	V	E

执行“LWL R2, 0x104”后

R2	D	A	V	E

初始值

M[200]	E			
M[204]		D	A	V
R2	J	O	H	N

执行“LWR R2, 0x203”后

R2	J	O	H	E

执行“LWL R2, 0x206”后

R2	D	A	V	E

5. 原子操作要检查**“操作是否完成”**：如果完整完成/无其他干扰，则跳出循环，结束程序；如果有返回错误/其他干扰，则返回标签再次运行。

第五章 静态流水线

1. 五级流水：取指、译码、执行、访存、写回

   三类相关：数据相关、控制相关、结构相关

2. 无旁路：**不能使用前递解决数据相关。**因此，遇到数据相关需要等到WB之后才能在下一步对相同的寄存器进行操作

   在进行循环操作时，要确定**循环出口**，题目中利用R3存放出口值，与R1进行比较。**分支指令需要延迟一拍**，所以在分支指令后可以放一条指令来消除冗余，这条指令依然会被执行。

   计算循环需要的时钟周期时，把下次循环的第一条指令 `I` 画到时空图，若一共 N 个循环，则前 $N-1$ 次的周期数为指令I之前的周期数 M_1，最后一次为单词循环的完整周期数 M_2。
   $总周期数 = (N-1) * M_1 + M_2$

假设此处的浮点运算均为单精度浮点运算。数组的长度为 N（$N<2^{15}-1$）。

```
        ADDIU   R3, R0, N        ; N
        SLL     R3, R3, 2        ; N*4
        ADDU    R3, R1, R3       ; R3=R1+N*4
Loop:   LWC1    F1, 0(R1)        ; load X(i)
        LWC1    F2, 0(R2)        ; load Y(i)
        MUL.S   F3, F0, F1       ; a*X(i)
        ADD.S   F4, F2, F3       ; a*X(i)+Y(i)
        ADDIU   R1, R1, 4        ; i++ for X
        ADDIU   R2, R2, 4        ; i++ for Y
        BNE     R1,R3, Loop
        SWC1    F4, -4(R1)       ; store to X(i)
```

第六章 动态流水线

1. 保留站：指令有序进入保留站，在保留站内等待相关被解决后乱序发射至功能部件进行执行

   重命名寄存器：通过对同一个逻辑寄存器重命名成多个不同的物理寄存器，可以解决WAW相关，并且避免伪RAW相关

   Reorder buffer：指令不论何时写回寄存器，它在reorder buffer中退出的顺序必须符合程序。这保证了乱序执行下，中断能有精确现场

类型	含义	是否是真相关
RAW	读后写	✅ 真相关（数据依赖）
WAR	写后读	❌ 假相关
WAW	写后写	❌ 假相关

2. **在Tomasulo算法中**：通过寄存器重命名技术，消除了`WAR`和`WAW`相关。在Tomasulo算法中，寄存器重命名的功能是通过保留站来实现的。当指令发射到保留站时，它的目标寄存器对应保留站号，消除了WAR相关和WAW相关。如果指令的某操作数没有准备好，在它的保留站中的该操作数的位置上标记的是产生该操作数的指令的保留站号。只有当操作数都准备好了，指令才能执行，保证了RAW相关

   在记分板技术中：在指令发射阶段，如果存在WAW相关，则停止发射该指令和任何后继指令。如果指令的源操作数都准备好了，则计分板会通知相应功能部件读取该操作数并开始执行该指令。这样保证了RAW相关。在指令写回阶段，如果存在WAR相关，则停止写回

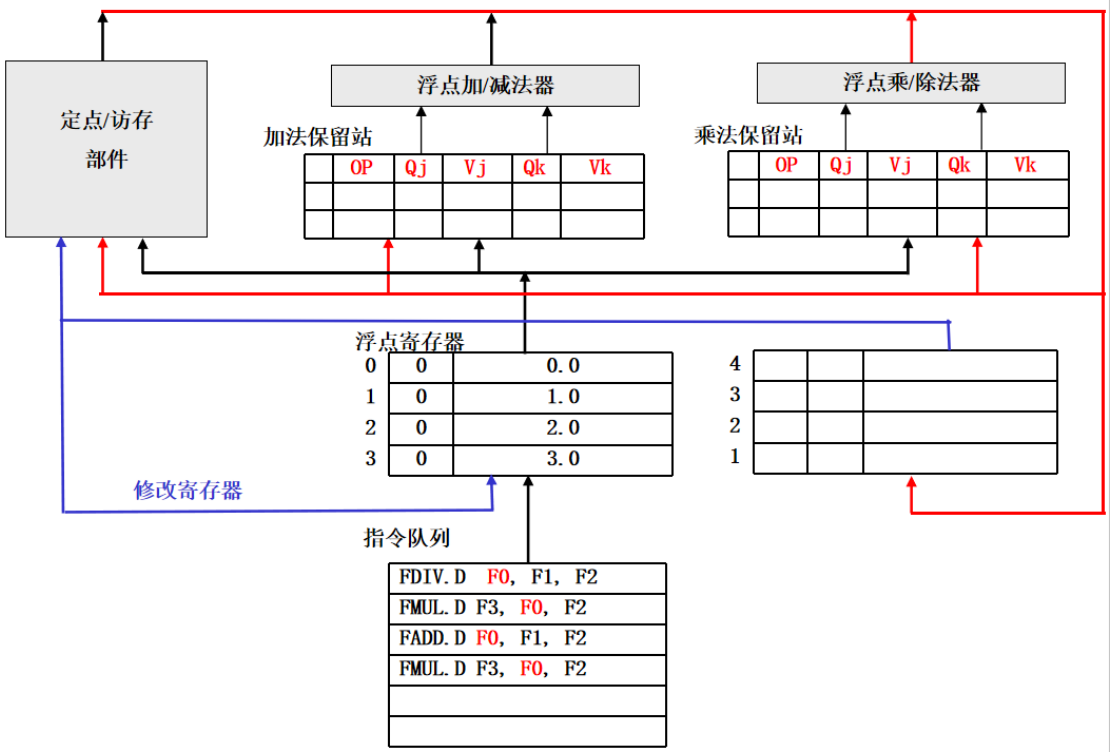
3. **精确例外**：就是指在处理例外的时候，发生例外指令之前所有的指令都已经执行完了，例外指令后面的所有指令都还没有执行（严格的说是没有产生执行效果，即修改处理机状态)

   实现精确例外处理的办法：重排序缓存(Reorder Buffer，简称ROB) 通过将指令执行结果暂存于按程序顺序排列的缓冲区，并仅在提交阶段顺序地更新体系结构状态，在发生异常时丢弃异常指令及其后的所有指令结果，从而保证处理器状态等价于顺序执行到异常指令为止，实现精确例外。**本质上是乱序执行，顺序提交**

4. （1）循环展开：**根据题目**确定冲突指令间的最小指令数，通过多循环进行填充，并**消除分支指令后的** `NOP`

   （2）软件流水：a. 画出数据流图，判断数据冲突；对于循环中自增/测试/分支指令，一般无数据冲突，**数据冲突只存在加载/运算/存储操作中**；b. 依据单循环的执行顺序**初步**确定填充/主体/排空；c. 尽量从第一组最后一个开始，划归循环主体。在划分循环主体时，要**①注意寄存器重命名**（可以通过填充流水确定）；**②同组内的数据冲突**；③同组内如果被划归到循环主体，要顺序执行；**④偏移量值的计算**（要进行溯源确认，同时记得自增后偏移量的改变）；**⑤如果出现“未用但要覆盖的情况”，要把覆盖的指令放置到循环主体中**；d. 循环主体添加自增/测试/分支指令，并**消除分支指令后的** `NOP`

5. 在填写寄存器结果时的注意事项：（1）保留站依次是：指令，操作数1是否就绪（就绪为 `0`，**否则为需要等待的指令编号**），操作数1的值（未就绪则空），操作数2是否就绪，操作数2的值；（2）**要着重注意指令队列中数据相关的指令**；（3）指令操作流程为：指令队列发射 — 写入 ROB — 写入保留站 — 从寄存器中取数并占用寄存器 — **运算结束后写回 ROB** — 按顺序写回浮点寄存器



第七章 多发射数据通路

3. RAW/WAR/WAW只针对寄存器，与内存同时访问可能产生的相关称为**“访存相关”**。两条指令能否同时发射取决于是否存在**不可消除的相关**
4. 双发射的模式是同时采用两条指令线，**一条定点指令线只用于加载/自增/存储，另一条浮点指令线用来运算。**
5. 每条指令在重命名阶段分配一个新的物理寄存器，并在提交阶段释放旧的物理寄存器。
   在流水线满负荷工作时，从重命名到提交共 t_2 拍，每拍发射 n 条指令，因此同时存在 $n \times t_2$ 条尚未提交的指令，占用 $n \times t_2$ 个物理寄存器。此外，m 个逻辑寄存器的当前状态需要 m 个物理寄存器保存。因此，最少需要的物理寄存器数为 $n \times t_2 + m$

第八章 转移预测

1.

$$\text{BHT 位数} = 2^6 \times 9 = 64 \times 9 = 576 \text{ bits}$$

$$\text{PHT 位数} = 2^{8+9} \times 2 = 262144 \text{ bits}$$

BHT：用 PC 低 6 位，选出一条“分支对应的历史寄存器”（9 位）；

PHT：用 **PC 低 8 位**（区分不同分支）加上 **9 位历史模式**，查 2 位计数器预测 Taken / Not Taken

表大小 = 2^(索引位数) × 每项位宽

2. 确定 “表的张数”，看：**GHR 用了几位来选表**（记为 k）。$\#Tables = 2^k$

   确定 “每张表的项数”，看：**索引到 PHT 用了几位**（来自 PC 位或分支编号，记为 p）。$\#Entries_per_table = 2^p$

   算总存储位数，每个 entry 通常是 **2-bit 饱和计数器**。$\text{PHT bits} = 2^k \times 2^p \times 2$。如果还有 BHT/BHR 等，也需要加上。

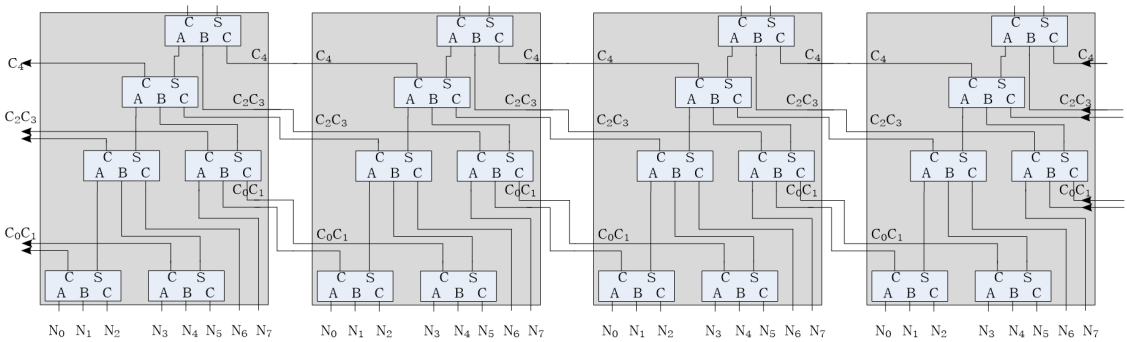
3. BTB命中且精确无额外开销，不命中会产生额外开销，命中且错误也会产生额外开销
4. 软件流水不能减少“分支出现的次数”，但能减少循环里的指令数； 循环展开可以减少分支出现的次数，但会让代码体积变大。

第九章 功能部件

2. 十六位串行进位加法器：从 c_0 传到 c_{16} 要过 16 级：$\text{delay}(c_{16}) = 16 \times 2T = 32T$，和位 s15 的延迟：最高位和位通常为$s_{15} = p_{15} \oplus c_{15}$，等到 c_{15} 计算出结果需要 $30T$，再做一次 XOR（$3T$），所以$\text{delay}(s_{15}) = 30T + 3T = 33T$

   先行进位加法器：产生 p/g（$2T$）→组 P/G（$2T$）→组间进位（$2T$）→组内进位（$2T$）→全加器求和（$3T$），**共** $11T$

3. 华莱士树的本质是将相同权重的数相加，**如果有8个16位二进制数相加，则有16棵华莱士树，因为有16位不同权重的数**；低位的华莱士树计算的进位值将传入高位的华莱士树；**每棵华莱士树有8个输入，因为相同权重的值有8个。**华莱士树之间是并行的，**延迟只与层数直接相关**（本质上是和输入计算的值数量有关），有几层就有几个全加器的延迟。8个4位二进制数相加的华莱士森林如图：



4. $[x]_{补} = 2^{n+1} + x$
5. CLA的Verilog代码

```
// ==========================================================
// 模块1：4位 CLA 子模块
// ==========================================================
module cla_4 (
    input  [3:0] a, b,   // 操作数
    input        cin,     // 进位输入
    output [3:0] sum,     // 和
    output       P, G     // 组传递、组生成
);

    wire [3:0] p, g;
    wire [3:0] c;

    // 1. 位级 p、g 计算
    assign g = a & b;
    assign p = a ^ b;

    // 2. 组 PG 计算
    assign P = &p;
    assign G = g[3] | (p[3] & g[2])
                    | (p[3] & p[2] & g[1])
                    | (p[3] & p[2] & p[1] & g[0]);

    // 3. 组内进位计算
    assign c[0] = cin;
    assign c[1] = g[0] | (p[0] & cin);
    assign c[2] = g[1] | (p[1] & g[0]) | (p[1] & p[0] & cin);
    assign c[3] = g[2] | (p[2] & g[1])
                       | (p[2] & p[1] & g[0])
                       | (p[2] & p[1] & p[0] & cin);
```

```
    // 4. 求和
    assign sum = p ^ c;
endmodule
// ========================================================
// 模块2：16位 CLA 顶层
// ========================================================
module cla_16 (
    input  [15:0] A, B,
    input         Cin,
    output [15:0] Sum,
    output        Cout
);

    // 1. 中间信号
    wire [3:0] P, G;   // 4 个 4-bit 组
    wire [4:0] C;      // 组进位（C0~C4）

    assign C[0] = Cin;

    // 2. 实例化 4 个 4-bit CLA
    cla_4 u0 (A[3:0],    B[3:0],    C[0], Sum[3:0],    P[0], G[0]);
    cla_4 u1 (A[7:4],    B[7:4],    C[1], Sum[7:4],    P[1], G[1]);
    cla_4 u2 (A[11:8],   B[11:8],   C[2], Sum[11:8],   P[2], G[2]);
    cla_4 u3 (A[15:12],  B[15:12],  C[3], Sum[15:12],  P[3], G[3]);

    // 3. 组间进位计算
    assign C[1] = G[0] | (P[0] & C[0]);
    assign C[2] = G[1] | (P[1] & C[1]);
    assign C[3] = G[2] | (P[2] & C[2]);
    assign C[4] = G[3] | (P[3] & C[3]);

    assign Cout = C[4];
endmodule
// ========================================================
// 模块3：24位 CLA 顶层
// ========================================================
module cla_24 (
    input  [23:0] A, B,
    input         Cin,
    output [23:0] Sum,
    output        Cout
);

    // 1. 中间信号
    wire [5:0] P, G;   // 6 个 4-bit 组
    wire [6:0] C;      // 组进位（C0~C6）

    assign C[0] = Cin;

    // 2. 实例化 6 个 4-bit CLA
    cla_4 u0 (A[3:0],    B[3:0],    C[0], Sum[3:0],    P[0], G[0]);
    cla_4 u1 (A[7:4],    B[7:4],    C[1], Sum[7:4],    P[1], G[1]);
    cla_4 u2 (A[11:8],   B[11:8],   C[2], Sum[11:8],   P[2], G[2]);
    cla_4 u3 (A[15:12],  B[15:12],  C[3], Sum[15:12],  P[3], G[3]);
    cla_4 u4 (A[19:16],  B[19:16],  C[4], Sum[19:16],  P[4], G[4]);
    cla_4 u5 (A[23:20],  B[23:20],  C[5], Sum[23:20],  P[5], G[5]);
```

```
    // 3. 组间进位（统一格式: G + P·C）
    assign C[1] = G[0] | (P[0] & C[0]);
    assign C[2] = G[1] | (P[1] & C[1]);
    assign C[3] = G[2] | (P[2] & C[2]);
    assign C[4] = G[3] | (P[3] & C[3]);
    assign C[5] = G[4] | (P[4] & C[4]);
    assign C[6] = G[5] | (P[5] & C[5]);

    assign Cout = C[6];
endmodule
```

第十章 高速缓存

1. 页与块的关系

```
|          页内偏移          |
| block index | block off |
```

页的大小决定页内偏移的大小，页内偏移包含块地址和块偏移，块地址与块的数量有关（多路组的情况下要平分），块内偏移与快的大小有关。

块地址与块偏移组成的但是在页内偏移之外的位数，是需要页着色的位数。“**由于页是内存映射的最小单位，cache 块不会跨越页边界。这种层级关系使得物理页号中的部分位参与 cache 索引，从而引入了 page coloring 的需求。**”

① 直接相联（1-way）

cache 组数

$$\frac{64\text{KB}}{32\text{B}} = 2048 = 2^{11}$$

👉 index 位数 = **11 位**

index 位范围

- block offset = [4:0]
- index = **[15:5]**

页内偏移是 [11:0]
👉 index 中 **超出页内偏移的部分**是:

```csharp
[15:12]
```

共 **4 位**

📌 这 4 位:

- 决定 cache 组
- 又不在页内偏移里
- OS 通过选物理页号可以控制

✅ **所以需要 4 位 page coloring**

3. 计算缺失率时，需要拉平到同一基线，每 `1000` 次访存有 $1000 \times \frac{26}{74} = 351$ 条 `Load/Store` 指令，因此总周期数有 `1351` 个

第十一章 存储管理

1. TLB refill 异常：TLB 中没有该页的映射，OS 查页表，把映射填入 TLB，**次数与TLB页数有关（相应的页大小与内存总量）**；TLB invalid 异常（Load / Store）：页表项存在，但 **TLB 项无效**，常见于页面第一次被访问或页被换出后又访问，**次数与TLB项数有关（相应的取决于一项有几个连续的虚拟页）**
2. 物理内存 64GB 需要 36 位物理地址；虚拟地址为 64 位，采用虚拟地址标记需额外区分进程，Tag 中加入 8 位 ASID。由于 index/offset 低位在两种标记方式下相同而相互抵消，Tag 额外位数为 $(64-36)+8=36$ 位，且不随块大小变化。

第十二章 多处理器系统

6. MSI 是最基础的一种 **监听式（snooping）一致性协议。**

状态	含义
M (Modified)	该 Cache 行被修改过，**只存在于本 CPU Cache 中**，主存是旧的
S (Shared)	数据是干净的，可被多个 CPU Cache 共享
I (Invalid)	Cache 中该数据无效

MSI 协议通过 **写独占、读共享、写使他人失效、修改需写回** 的规则，保证多 CPU Cache 中的数据始终一致。**写独占时，其他都要变为I；读共享时，如果其他存在独占，则要退化为共享。**